Building the /Qanalyse Skill

A Walkthrough of Multi-Agent Investment Analysis with Quartr MCP

Date: 2026-03-30

Tool: Claude Code (Claude Opus 4.6)

Duration: Single conversation session

What We Built

A multi-agent investment analysis skill (/Qanalyse) that orchestrates 10 specialist AI sub-agents in parallel, combining web research with deep primary-source analysis from Quartr's transcript, document, and financial data libraries. It produces a comprehensive investment research report with audio narration.

Starting Point: The Existing /analyse Skill

We already had a working /analyse skill - a Claude Code "command" (a markdown file at ~/.claude/commands/analyse.md) that:

- Runs **8 specialist sub-agents** in parallel, each analysing a different dimension of a company
- Each agent uses web search to find current information
- An orchestrator synthesises the results into a final report
- Post-processing generates audio (via OpenAI TTS) and deploys to a phone app via git push

The 8 original agents:

1. **Business History** - founding story, pivots, structural changes
2. **Management & Incentives** - leadership, compensation, capital allocation track record
3. **Competitive Landscape** - moats, market position, threats
4. **Financial Characteristics** - revenue model, margins, returns on capital
5. **Red Flags** - accounting concerns, governance risks, bear case
6. **Investment Checklist** - 12-point YES/NO evaluation
7. **Learning Resources** - podcasts, books, articles, earnings calls
8. **Corporate Culture** - founder imprint, talent, innovation, ethics

Each agent was guided to produce ~400 words of concise analytical prose.

The Enhancement: Integrating Quartr MCP

What is Quartr?

Quartr is a financial data platform with an MCP (Model Context Protocol) integration for Claude. MCP tools let Claude interact with external services directly - calling APIs, fetching data, reading documents - all within the conversation.
The Quartr MCP provides tools for:
- **Company search and profiles** - structured company data, GICS classification
- **Financial statements** - income statement, balance sheet, cash flow (quarterly and annual)
- **Events** - earnings calls, investor days, capital markets days, AGMs
- **Documents** - transcripts, slide decks, annual reports (10-K, 10-Q, 20-F), shareholder letters, ESG reports, press releases
- **Full-text search** - search across all transcripts, slides, and reports
- **Event/document summaries** - AI-generated summaries
- **Related companies** - peer identification

Design Decisions

1. Phase 0: Structured data gathering before agent dispatch
The biggest architectural decision was adding a "Phase 0" where the orchestrator fetches foundational data from Quartr *before* dispatching any sub-agents. This means every agent starts with real structured data rather than relying solely on web search.
Phase 0 fetches:
- Company profile (via search_companies -> get_company)
- Annual and quarterly financial statements (via get_financials)
- Recent events list with earnings call summaries (via list_events -> get_event_summary)
- Full document catalogue (via list_documents)
- Peer companies (via list_related_companies)

This is compiled into a "Quartr Context Blob" that gets passed to each agent with their specific instructions.
Why this matters: Without Phase 0, each of the 10 agents would independently need to search for the company, discover its Quartr ID, and fetch data - duplicating work 10x and wasting time. By centralising the data gathering, agents start immediately with rich context.
2. Two new specialist agents
We added two agents that specifically exploit Quartr's primary-source access:
Agent 9: Earnings Call Deep Dive reads 3-4 full earnings call transcripts and analyses:
- Narrative evolution across quarters
- Guidance vs delivery tracking
- Analyst concerns from Q&A (what they keep pushing on)
- Management language signals (hedging, confidence markers)
- Key reveals - specific quotes where management said something significant
- Q&A dynamics - which analysts, what debates

This is the single biggest value-add from Quartr integration. Web search gives you commentary *about* earnings calls; Quartr gives you the actual words management used. The difference is enormous for assessing management quality and credibility.
Agent 10: Strategic Documents Analysis reads investor presentations, shareholder letters, and capital markets day materials to analyse:
- Strategic priorities and their evolution
- Capital allocation framework (rhetoric vs numbers)
- Long-term targets and their credibility
- Narrative vs reality gaps
- TAM and growth framing
- Risk acknowledgement (or lack thereof)

These documents are management's most carefully crafted communications - how they frame the business for sophisticated investors reveals strategic thinking that doesn't show up in press coverage.
3. Enhanced existing agents with targeted Quartr integration
Each original agent got specific Quartr enhancement instructions:

Agent	Key Quartr Enhancement
Business History	Annual reports for business descriptions; document search for acquisition/pivot menti...

Management & Incentives	Read actual earnings call transcripts; search for management names and compensation d...
Competitive Landscape	Peer companies list; search transcripts for competitor mentions; investor presentatio...
Financial Characteristics	**Actual structured financial data** instead of web-search approximations; 10-K/10-Q ...
Red Flags	Search transcripts for risk terms ("litigation", "impairment", "restatement"); 10-K r...
Investment Checklist	Full context blob for grounding each verdict in specifics
Learning Resources	Reference Quartr events as primary sources; highlight notable investor days
Corporate Culture	Search transcripts for culture terms; ESG/sustainability reports for workforce metric...

4. Longer reports
Report length was increased from ~400 words to ~600 words minimum per agent, with explicit permission to expand to 800-1000+ words where information is rich. The synthesis section went from ~500 to 750-1000 words. The reasoning: with access to primary sources, agents have genuinely more to say - artificial compression would waste the better data.

What the Skill File Looks Like

A Claude Code command is a markdown file with YAML frontmatter. The key parts:

```
---
description: Run a Quartr-enhanced multi-agent investment analysis on a company
argument-hint: [company name]
allowed-tools: Bash, Read, Write, Edit, Agent, Glob, Grep, WebSearch, WebFetch,
  ToolSearch, mcp__claude_ai_Quartr__search_companies,
  mcp__claude_ai_Quartr__get_company, mcp__claude_ai_Quartr__get_financials,
  # ... all Quartr MCP tools listed
---
```

- description - shown in the skill list
- argument-hint - tells the user what to pass (e.g., /Qanalyse Apple)
- allowed-tools - whitelists which tools the orchestrator can use (must include all Quartr MCP tools)

The body is a prompt that instructs Claude on exactly how to orchestrate the analysis. It uses $ARGUMENTS as a placeholder for the company name the user provides.

How Sub-Agents Work

When the orchestrator runs, it uses Claude Code's Agent tool to spawn sub-agents. Each sub-agent:

- Runs as an independent Claude instance with its own context
- Gets a detailed prompt including the Quartr context data and specific instructions
- Has access to all tools (WebSearch, Quartr MCP, etc.)
- Returns its analysis as a single message to the orchestrator
- Runs in parallel with all other agents

The orchestrator waits for all 10 to complete, then writes its own synthesis incorporating insights from all of them.

The Full Pipeline

```
User runs: /Qanalyse [Company Name]

Phase 0: Orchestrator fetches Quartr data (~30 seconds)
    |-- search_companies -> get companyId
    |-- get_company (profile)
    |-- get_financials (yearly)
    |-- get_financials (quarterly)
    |-- list_events (recent)
    |-- list_documents (catalogue)
    |-- list_related_companies (peers)
    `-- get_event_summary x 3-4 (recent earnings calls)

Phase 1: 10 agents run in parallel (~2-5 minutes)
    |-- Agent 1: Business History
    |-- Agent 2: Management & Incentives
    |-- Agent 3: Competitive Landscape
    |-- Agent 4: Financial Characteristics
    |-- Agent 5: Red Flags
    |-- Agent 6: Investment Checklist
    |-- Agent 7: Learning Resources
    |-- Agent 8: Corporate Culture
    |-- Agent 9: Earnings Call Deep Dive      <- NEW
    `-- Agent 10: Strategic Documents          <- NEW

Phase 2: Orchestrator writes synthesis (~30 seconds)

Phase 3: Save report to ~/investment-research/

Phase 4: Audio & deployment
    |-- TTS generation (OpenAI API)
    |-- M4A compression (for WhatsApp sharing)
    |-- Manifest update
    |-- Git commit & push
    `-- Available on phone app
```

Ideas Not Yet Implemented

- **Dedicated Valuation Agent** - the checklist has one line on valuation, but with structured financials we could do DCF, comps, and historical multiple analysis
- **Peer Comparison Agent** - use list_related_companies + get_financials for multiple companies to build a comparative table
- **Management Tone Tracking** - quantitative analysis of language patterns across many quarters of transcripts
- **Automated Watchlist Integration** - add analysed companies to a Quartr watchlist for ongoing monitoring

Key Takeaways for Building Multi-Agent Skills

1. **Centralise shared data gathering** - fetch common data once in a setup phase rather than having each agent duplicate the work
2. **Give agents primary sources, not just search** - the quality difference between "search the web for earnings info" and "read the actual transcript" is substantial
3. **Be specific in agent prompts** - tell each agent exactly which tools and data types are most relevant to their focus area
4. **Let agents go deep when warranted** - artificial word limits waste good data access; set minimums but allow expansion
5. **Parallel execution is key** - 10 agents running simultaneously takes roughly the same wall-clock time as 1; sequential would be 10x slower
6. **The orchestrator's synthesis matters** - it's the only place that sees all 10 analyses and can identify tensions, contradictions, and cross-cutting themes
7. **MCP tools in the frontmatter** - if the orchestrator needs to call MCP tools directly, they must be listed in allowed-tools
8. **Sub-agents can discover tools** - by instructing agents to use ToolSearch, they can fetch schemas for MCP tools they need at runtime